Stage 0, in full:
the environment gate before any model run

2026-11-01

This is the complete protocol of Stage 0 of the experiment plan, and of Stage 0b, which pushes its third gate through the real training pipeline. It is written so that a reader who has the paper but not the scripts can see exactly what is done to obtain every controller, every number and every verdict, and can reproduce them. Stage 0 was first run on 2026-10-24 against an earlier version of the environment; it is to be re-run against the current one, and this document holds the protocol for that re-run with the results left blank (§9). The paper's Stage 0 subsection will be a summary of this document and its appendix will carry the essentials; §13 says which parts go where.

Terms. A *controller* is a Python function `policy(state)` that returns one action per step; the paper also calls it the policy or the final run. A *batch* is one episode of the crystallizer: one seed fixes its start temperature, its target temperature and its seed loading. The *hidden limit* $\sigma_{\max}$ is the level of supersaturation at which the batch showers and is ruined. A *blind* controller does not know $\sigma_{\max}$; an *informed* controller was tuned for one known value of it; the *oracle* is the controller that rides exactly at $\sigma_{\max}$ and sets the time-optimal reference. *Fast success* means the batch succeeds within 1.25× the oracle's step count for that batch and limit. *Normalised time* is the oracle's steps divided by the controller's own steps, and zero on failure. An *anchor* is one fixed value of $\sigma_{\max}$ at which the informed side is tuned and both sides are reported.

1 What Stage 0 is, and what it is not

Stage 0 does not use a language model. No prompt is written, nothing is told to anything, and no text is generated. Every controller in Stage 0 is a short formula written by hand, with between one and ten numerical knobs, and the knobs are set by a simple search on the environment. The stage runs on a laptop CPU in a few minutes.

Its purpose is to answer one question before any model run is spent: *does knowing the hidden limit buy anything that a controller without it cannot get from the readings of its own batch?* The whole study rests on a yes. The paper's thesis is that a first-turn probe run, which finds the limit by crossing it on a throwaway batch, is worth credit because the final controller needs the limit and cannot learn it in-episode. If a blind controller could be nearly as fast as an informed one, the probe would be worthless, every training arm would learn to skip it, and the credit assignment the paper proposes would have nothing to assign. The earlier slosh-tank environment failed exactly this gate and was dropped; the crystallizer replaced it.

The gate therefore sits before everything else in the ladder. Stage 0b then checks that the solution Stage 0 found survives the real execution path. Only after both pass is the brief put in front of the model (Stage 1).

2 The environment, as much as Stage 0 needs

A stirred tank holds one kilogram of solvent with dissolved solute and a known mass of small seed crystals. A jacket cools or heats it. The controller sets the jacket rate once every 20 s step as a fraction of the jacket's limit of 1.5 °C/min, so one step of full cooling lowers the temperature by 0.5 °C. The batch starts saturated at 55–65 °C and must reach a target of 18–25 °C. Cooling raises the supersaturation σ, the fraction by which the dissolved amount exceeds what the liquid can hold at the current temperature; the seeds grow and consume it. Success means reaching the target temperature (within 0.25 °C) with $\sigma \le 0.02$. If σ ever exceeds the hidden limit $\sigma_{\max}$, drawn once per task from $U[0.08, 0.25]$, a shower of new nuclei ruins the batch and the episode ends in failure. The horizon is 600 steps. The reward is -0.3 per step, $+120$ on success and -80 on failure. The full dynamics and constants are in the paper's crystallizer appendix.

The hidden limit appears nowhere except in the shower test. Two batches with different limits produce identical readings under identical actions until one of them showers, so no in-episode estimator exists. The fastest safe recipe holds σ just below $\sigma_{\max}$ and accelerates as the crystals grow (classical programmed cooling). A controller that does not know the limit must hold σ at the bottom of the range to be safe on every batch, and is then slower than the oracle, the more so the higher the batch's limit.

The environment changed on 2026-10-26, after the first Stage 0 run. Table 1 lists the differences that matter here; the re-run uses the current column throughout.

	2026-10-24 (first Stage 0 run)	current (the re-run)
σ in the state	no; from c and T via the solubility curve	yes, a reading
Shower test applied to	mid-step σ, before growth	end-of-step σ, the reported value
Time weight (reward per step)	-0.1	-0.3
Success bonus, failure penalty	$+120$, -80	$+120$, -80
Error floor for a crashing controller	-200	-300
Oracle	tracking controller with setpoint $\sigma_{\max}$	bisection on the end-of-step σ
What the model is given	environment source	a brief in words

Table 1: The two environment versions.

3 What a controller is in Stage 0

Every Stage 0 controller works the same way: it computes a desired temperature T_{des} for the end of the next step, and returns the jacket fraction that gets there, clipped to the jacket's limit,

$$a = \operatorname{clip}\left(\frac{T_{\text{des}} - T}{0.5\,^{\circ}\text{C}},\, -1,\, 1\right).$$

The environment only ever receives a; the desired temperature is an intermediate inside the controller's own code, convenient because the families are stated in temperature. They differ only in how they choose T_{des}. Two of them are feedback controllers on the concentration; three are open-loop time profiles. The feedback building block is *tracking*: given a setpoint σ_t, the temperature at which the current concentration c is exactly σ_t above saturation is $T_{\text{track}} = T_{\text{sat}}\big(c/(1+\sigma_t)\big)$, with T_{sat} the inverse of the solubility curve, and never below the target temperature. Hand-written code may use the solubility curve, since it is the environment's own function. In the current environment σ is also a reading, so the block can equally be written on the reading; the search tunes the setpoint either way.

Each knob p_k lives in $[0, 1]$ and is mapped to a physical range inside the family (Table 2).

Family	Knobs	Physical parameters	What it does
track	1	$\sigma_t = 0.02 + 0.33\,p_0 \in [0.02, 0.35]$	Hold the supersaturation at σ_t: each step, move toward $T_{\text{track}}(\sigma_t)$. Feedback on c.
linear	1	rate $= 0.02 + 1.48\,p_0$ °C/min	Cool at one constant rate until the target. Open loop.
power	2	$\tau = 10 + 190\,p_0$ min; $n = 0.5 + 3.5\,p_1$	Follow $T(t) = T_0 - (T_0 - T_{\text{target}})\,\min(1, t/\tau)^n$. Open loop.
hybrid	3	τ, n as above; cap $= 0.02 + 0.33\,p_2$	The power profile, but never below $T_{\text{track}}(\text{cap})$, so σ never exceeds the cap. Feedback on c.
pwl	10	four knot times $p_0..p_3$ on the horizon fraction $t/600$; six levels $p_4..p_9$	Piecewise-linear profile: $T_{\text{des}} = T_{\text{target}} + f(t)\,(T_0 - T_{\text{target}})$ with f linear between knots. Open loop.

Table 2: The five hand-written controller families. T_0 is the batch's start temperature, remembered at the first call.

4 The two sides and the hidden-value configurations

Two hidden-value configurations are run. The *range* configuration draws $\sigma_{\max}$ for each batch from $U[0.08, 0.25]$ as a fixed function of the seed, and reports at three anchors, 0.08, 0.165 and 0.25. The *set* configuration cycles the limit through $\{0.08, 0.12, 0.18, 0.25\}$ by seed, and reports at those four values. The range configuration is the one the training tasks use; the set configuration is a check that the result does not depend on the limit being continuous.

The **blind side** gets one controller per family per configuration. Its knobs are tuned on 48 search batches whose limits vary as the configuration says, so the single controller must do well across all of them without ever being told which limit a batch has. Nothing in its inputs depends on $\sigma_{\max}$.

The **informed side** gets one controller per family per anchor. Its knobs are tuned on 48 search batches that all share that anchor's limit. It "knows" the limit only in this sense: it was tuned for it. It reads the same state as the blind side.

Both sides use the same five families, the same search budget, the same 48 search seeds, the same 200 evaluation seeds, and the same objective, the mean environment reward, which is what the later reinforcement learning would maximise. The range configuration therefore tunes 5 blind controllers and $5 \times 3 = 15$ informed ones; the set configuration 5 and $5 \times 4 = 20$.

5 How the knobs are tuned

The search is random sampling followed by hill climbing, identical for every family and both sides.

1. Draw 300 knob vectors uniformly in the unit cube of the family's dimension.
2. Run each on the 48 search batches (seeds 0 to 47) and record the mean reward. A controller that raises an exception on a batch scores the error floor on that batch.
3. Keep the best 6.
4. For 120 rounds: nudge each survivor by Gaussian noise with a scale that shrinks from about 0.16 to 0.01 over the rounds, clip to the cube, evaluate the nudged vector on the same 48 batches, and keep it in place of its parent if it scores higher.
5. Return the best of the 6.

Random knob vectors	300
Survivors into hill climbing	6
Hill-climbing rounds	120
Noise scale per round	$0.15\,(1 - r/120) + 0.01$
Search batches	seeds 0 to 47
Candidate evaluations per controller	$300 + 6 \times 120 = 1020$, each over 48 batches

Table 3: Search budget, the same for every family, both sides, both configurations.

6 How a controller is scored

Tuned controllers are scored on 200 fresh batches, seeds 1000 to 1199, that the search never saw. Under the range configuration each fresh seed carries its own limit; at an anchor all 200 share the anchor's limit. Three numbers are averaged over the 200 batches.

- **Success**: reached the target temperature with $\sigma \leq 0.02$, no shower, no seed loss, within 600 steps.
- **Fast success**: success and steps $\leq 1.25\times$ the oracle's steps on that batch at that limit.
- **Normalised time**: oracle steps divided by own steps; zero on failure.

The **oracle** defines the time-optimal step count for every (batch, limit) pair. In the current environment it is the most cooling at each step such that the end-of-step σ stays at or below $\sigma_{\max}$, found by bisection on a copy of the environment; it is run once per pair and cached. (In the 2026-10-24 environment it was the tracking controller with its setpoint a hair below $\sigma_{\max}$, the same idea applied to the mid-step shower test.) The fast-success threshold of $1.25\times$ was fixed before the first search ran and is kept.

Fast success is the headline number because raw success is uninformative here: a cautious blind controller succeeds on every batch by being slow.

7 The three gates

All three were fixed before the first search was run and are kept for the re-run. The gate is one verdict per configuration; G1 and G2 come from the search, G3 from the hand-written two-turn solution of §8.

Gate	Condition	What it checks
G1	informed fast success $\geq 90\%$ at every anchor	The task is doable when the limit is known: the families are rich enough to ride it.
G2	blind fast success at least 40 points below the informed mean over the range	The limit matters: not knowing it costs speed that no blind controller recovers.
G3	hand-written probe run plus controller $\geq 95\%$ fast success at every anchor	The gap is closable by probing: a probe on a throwaway batch recovers the limit well enough to ride it.

Table 4: The pre-registered gates.

G1 without G2 would mean the environment is easy without the limit, so the probe is pointless. G1 and G2 without G3 would mean the limit matters but cannot be found, so the probe is pointless for a different reason. All three together say the probe is the way through.

8 The hand-written probe run and controller

This is the one place in Stage 0 where something has the shape of what the model is later asked to do, except that a person wrote it. The probe run drives a throwaway batch with the same hidden limit as the scored batch and prints what it learned; the controller is then written from the printed number. In Stage 0 both run in-process; Stage 0b pushes them through the real sandbox (§11).

Why the probe has to cross the limit. The limit shows up only as the shower. A batch that never showers says nothing about it. So the probe cools until the batch showers, and the last reading before the shower is a lower bound on $\sigma_{\max}$ while the reading at the shower is an upper bound. Cooling at the jacket limit raises σ by about 0.01 per step, as the quench logs in the brief show, so one batch brackets the limit to about the rise of one step. Further batches can go quickly to the lower bound and then creep up in small increments until the shower, tightening the bracket.

The three-pass probe. Pass one cools at the jacket limit until the shower. Passes two and three reset, drive σ quickly to the lower bound, then raise the setpoint in eighths of the current bracket until the shower, and tighten the bracket each time. It prints the bracket. The version used in the re-run works from the σ reading alone and is listed in §11. The 2026-10-24 version, kept here for the record, computed the mid-step σ from the concentration and the solubility curve, because that is the value the shower test used then.

```
# Probe run: find the shower limit by crossing it. Three passes, each one batch.
cfg = env.config
def sat_T(conc):
    return saturation_temperature(conc, cfg)
def step_to_sigma(sig):
    """one step toward the temperature that puts the pre-growth supersaturation at sig"""
    T_des = max(sat_T(s["c"] / (1.0 + sig)), s["T_target"])
    return max(-1.0, min(1.0, (T_des - s["T"]) / (cfg.max_rate * cfg.dt / 60.0)))

lo, hi = 0.0, 0.40                      # bracket on sigma_max: safe below lo, shower above hi
for npass in range(3):
    s, _ = env.reset()
    target = lo                         # go quickly to the last safe level ...
    inc = (hi - lo) / 8.0               # ... then creep up in eighths of the bracket
    while True:
        c_prev = s["c"]
        a = step_to_sigma(target)
        if npass == 0 and lo == 0.0:
            a = -1.0                    # first pass: cool at the jacket limit
        s, r, done, _, _ = env.step(a)
        sig_pre = c_prev / solubility(s["T"], cfg) - 1.0   # the value the shower test used
        if done:
            if env.is_success():
                lo = max(lo, sig_pre); break
            hi = min(hi, sig_pre); break
        lo = max(lo, sig_pre)
        if abs(sig_pre - target) < 1e-4 or npass == 0:
            target = min(target + inc, hi)
print(f"SIGMA_MAX_LOW={lo:.5f} SIGMA_MAX_HIGH={hi:.5f}")
```

The controller (2026-10-24 version). The low end of the bracket is pasted in as `SIGMA_HAT`, and the controller tracks 98% of it, the tracking family of Table 2 with its setpoint set from the probe.

```
SIGMA_HAT = <the printed SIGMA_MAX_LOW>
MARGIN = 0.02
def policy(state):
    cfg = EnvConfig()
    T_des = saturation_temperature(state["c"] / (1.0 + SIGMA_HAT * (1.0 - MARGIN)), cfg)
    T_des = max(T_des, state["T_target"])
```

```
a = (T_des - state["T"]) / (cfg.max_rate * cfg.dt / 60.0)
return max(-1.0, min(1.0, a))
```

How much probing is needed. Three probe variants are compared on the range configuration, each followed by the same controller: the three passes above; one pass at the jacket limit, which stops at the first shower; and one pass that raises the setpoint by a fixed increment every step until the shower, at increments of 0.005 and 0.001. The question is whether a single batch is enough, and whether a slow creep that never crosses the limit at high limits can still bracket it.

9 Results

To be filled after the re-run. The re-run reports, in this order, all on the current environment:

1. **Blind side, range configuration.** One row per family: mean reward, success, fast success, normalised time, over the 200 fresh batches; and the setpoint or profile the winning family settled on.
2. **Informed side, range configuration.** One row per family, one column per anchor: fast success and mean reward.
3. **Gate verdicts.** One row per configuration (range, set): blind fast success over the range and at each anchor, blind normalised time and reward, informed fast success and reward at each anchor, the gap in percentage points, G1 and G2.
4. **G3 and the probe variants.** One row per variant: fast success, normalised time, maximum probe error $|\sigma_{\max} - \text{SIGMA_MAX_LOW}|$, and fast success at the top anchor.
5. **Reading of the result.** Whether the gap comes from the physics or from a narrow family or a withheld sensor; what the blind side's optimum is under the reward; whether it gambles.
6. **Stage 0b** (§11): one row per third of the limit range and one overall: fast success, normalised time with its minimum, reward, maximum probe error, and the blind cautious controller's fast success, normalised time and reward on the same batches; the probe-error distribution (median, ninetieth percentile, maximum); the gate verdict.

10 The environment change of 2026-10-26

On 2026-10-26 the prompt was replaced by a brief in words, without the environment's source or any equation. Three environment changes followed from that decision. Without the solubility equation a model cannot compute σ from c and T, so σ became a reading in the state, as an in-line concentration probe would give it. The shower test moved from the mid-step value, which the readings never show, to the end-of-step value that the state reports, so that what the model sees is what is tested. The time weight was set to 0.3 so that the probe is worth enough reward to be learned.

The first Stage 0 search ran before these changes and was not repeated after them. The re-run closes that gap: the search, the gates and Stage 0b are all run on the current environment. For the search script this means the oracle is the bisection one, the error floor is the current one, and the tracking building block may target the end-of-step σ; the families, the sides, the budget, the seeds and the gates are unchanged.

11 Stage 0b: the same solution through the real path

Stage 0 runs the probe and the controller in-process, with the environment's functions in scope. Stage 0b asks whether the solution survives the way training and evaluation actually execute code:

the probe source is pushed through the runner's turn executor, which runs it in a sandboxed subprocess against a probe-run environment that has the same hidden limit as the scored batch but a different batch and exposes only `reset`, `step`, the end flag, the failure reason and the outcome tests; its printed bracket is pasted into the controller template; and the controller is scored with the same function the trainer uses, on the scoring batch. Pre-registered gate: fast success $\geq 95\%$ with no probe failures. The blind cautious controller, which holds the reading at 0.077, is run on the same batches as the reference. The batches are the 36 of the task files plus fresh ones built by the task builder's rule.

From the brief alone. The probe and the controller use only what the brief offers: `env.reset()`, `env.step(a)`, the end flag, `env.failure_reason` and the σ reading. No solubility curve, no concentration arithmetic. This is the proof that the task is solvable with exactly what the model is told, and it is the version the re-run uses for G3 as well.

The probe keeps the three-pass structure. Pass one cools at the jacket limit and records the highest reading seen before the shower as the lower bound and the reading at the shower as the upper bound. Passes two and three go to the lower bound and creep up in eighths of the bracket, using a cautious guess of how much σ rises per step of full cooling to convert a setpoint into an action.

```
# Probe run: find the level of sigma at which this crystallizer showers, by crossing it.
lo, hi = 0.0, 0.40    # sigma_max is above lo (a reading seen safe) and below hi (a reading at a shower)
G = 0.016             # cautious guess at how much sigma rises per full-cooling step
for npass in range(3):
    s, _ = env.reset()
    if npass == 0:
        while True:         # first batch: cool at the jacket limit until the shower
            s, r, done, _, _ = env.step(-1.0)
            if done:
                break
            lo = max(lo, s["sigma"])
        if env.failure_reason == "shower":
            hi = min(hi, s["sigma"])
        continue
    target, inc = lo, (hi - lo) / 8.0      # later batches: go to the last safe level, then creep up
    while True:
        a = max(-1.0, min(0.0, -(target - s["sigma"]) / G))
        s, r, done, _, _ = env.step(a)
        if done:
            break
        lo = max(lo, s["sigma"])
        if s["sigma"] >= target - inc / 2:
            target = min(target + inc, hi)
    if env.failure_reason == "shower":
        hi = min(hi, s["sigma"])
print(f"SIGMA_MAX_LOW={lo:.5f} SIGMA_MAX_HIGH={hi:.5f}")
```

The controller holds the reading 0.003 under the level found. It needs memory, because the crystals consume supersaturation between steps and a controller that ignores that either undershoots or showers: it estimates each step how much the crystals ate, as what cooling should have added minus what the reading shows, smooths that estimate, and adds it back to the cooling demand. It never overshoots the target temperature.

```
SETPOINT = <the printed SIGMA_MAX_LOW> - 0.003
G = 0.014                       # about how much sigma rises per full-cooling step
mem = {"sigma_prev": None, "a_prev": 0.0, "eaten": 0.0}
def policy(state):
    sigma, T, T_target = state["sigma"], state["T"], state["T_target"]
    if mem["sigma_prev"] is not None:
        # what the crystals ate last step: what cooling should have added, minus what we got
        eaten = mem["sigma_prev"] - mem["a_prev"] * G - sigma
        mem["eaten"] = 0.5 * mem["eaten"] + 0.5 * max(0.0, eaten)
    if T <= T_target:
        a = 0.0
    else:
```

```
    a = max(-1.0, min(0.0, -(SETPOINT - sigma + mem["eaten"]) / G))
    a = max(a, -(T - T_target) / 0.5)      # do not overshoot the target temperature
mem["sigma_prev"], mem["a_prev"] = sigma, a
return a
```

Two simpler controllers written from the same number are run alongside it, to show what the memory buys: full cooling until the reading hits the setpoint and then hold, with a thin margin and with one step of margin; and proportional control on the reading without memory.

12 How to run

- **Stage 0 search and G3.** `scripts/stage0_gate_crystal.py`: the `search` subcommand with `--configs range:0.08-0.25 set:0.08,0.12,0.18,0.25` and the default budget (300 random, top 6, 120 climbing rounds); G3 with the `two-turn` subcommand, and the variants with `--probe one-pass` and `--probe creep --creep-step 0.005|0.001`. The script as committed records the 2026-10-24 environment (its oracle and its tracking block target the mid-step σ) and must be brought to the current one before the re-run, as §10 says. Logs go to `docs/stage0/` and JSON to `data/stage0/`, dated.
- **Stage 0b.** `scripts/stage0b_two_turn.py` with `--n-random 100`; it runs the readings-only probe and controller above through `execute_turn` and `score_sample` on the task files plus the fresh batches, and writes every per-batch row to `data/stage0b/`, dated; the compact result goes to `docs/stage0b/`.
- **Diary.** Each run gets a dated entry in `docs/findings.tex` with the commit, the command and the raw output location.

13 What goes where in the paper

This file stays as the full record. The paper's Stage 0 subsection should say, in about eight sentences: that no model is involved; that the controllers are hand-written formulas of five families with one to ten knobs, tuned by random search and hill climbing; that a blind side is tuned across limits and an informed side per limit, same families, same budget, same seeds, same objective; that controllers are scored on fresh batches by fast success against an oracle that rides the limit; the three gates; the verdict, with the blind and informed numbers once the re-run has them; and a pointer to the appendix. The Stage 0b subsection should say what the real path adds and that the solution is written from the brief alone, with its numbers.

The appendix should carry, from this file: Table 2 (families), Table 3 (budget), the scoring definitions and the oracle of §6, Table 4 (gates), the readings-only probe and controller listings of §11, and, once filled, the gate-verdict table, the probe-variant table and the Stage 0b table of §9. The 2026-10-24 listings and the per-family tables can stay here.